DIVISÃO DE TAREFAS — 4 INTEGRANTES

Sistema de Gestão & Aluguel de Equipamentos Esportivos

Projeto Final — Desenvolvimento de Aplicações Desktop (Electron + TypeScript)

Pessoa	Área	Principais entregas
1	Arquitetura e Banco de Dados	Projeto base, 10 tabelas com FKs, repositórios, IPC/preload
2	Regras de Negócio e CRUD	Entidades, CRUD de tudo, disponibilidade, multas, financeiro, manutenção
3	APIs e Segurança	CEP, clima, rotas, notificações; bcrypt, perfis, validação, Electron seguro
4	Interface (UI/UX)	Todas as telas, fluxo de reserva com alerta de clima, dashboard, instalador

Requisitos obrigatórios do professor

1. Banco relacional com no mínimo 6 tabelas relacionadas (FKs) — Pessoa 1
2. CRUD completo para todas as entidades — Pessoas 2 e 4
3. Consumo de API em 4 situações distintas — Pessoa 3
4. Segurança justificada — Pessoa 3 (com revisão de todos)
5. Código modularizado em classes (POO) — Todos
6. Electron com TypeScript, sem .js de lógica — Pessoas 1 e 4

Tarefas de todos os integrantes

- ☐ Cada pessoa faz commits próprios, pequenos e frequentes, com o seu nome/e-mail (o professor avalia o histórico por autor)
- ☐ Adicionar o usuário willcotil como colaborador do repositório no GitHub desde o início
- ☐ Usar uma branch por tarefa e abrir Pull Requests para a main; outra pessoa revisa
- ☐ Reunião rápida semanal para alinhar contratos de IPC/DTOs entre as Pessoas 1, 2, 3 e 4
- ☐ Usar os agentes de IA de .claude/agents/ e saber explicar cada decisão tomada com ajuda deles
- ☐ Relatório final + apresentação: cada pessoa escreve e apresenta a sua parte

Sugestão de cronograma

Semana 1	Pessoa 1: projeto base, banco e IPC • Pessoa 2: entidades e enums • Pessoa 3: login/bcrypt e configuração de segurança • Pessoa 4: tema, layout e roteador
Semana 2	Pessoa 1: repositórios • Pessoa 2: CRUDs • Pessoa 3: CEP e clima • Pessoa 4: telas de cadastro
Semana 3	Pessoa 2: reservas, multas e manutenção • Pessoa 3: rotas e notificações • Pessoa 4: reservas/devoluções e dashboard • Pessoa 1: apoio e integração
Semana 4	Todos: testes, correções, revisão de segurança, documentação, empacotamento e apresentação

Pessoa 1 — Arquitetura, Banco de Dados e Base do Projeto

Responsável: _____

Foco: Monta a fundação que todos os outros vão usar: projeto Electron + TypeScript, estrutura de pastas, banco relacional e a ponte IPC entre Main e Renderer.

Requisitos cobertos: Req. 1 (6+ tabelas com FKs), Req. 5 (POO), Req. 6 (Electron + TypeScript)

Agente de IA de apoio: `arquiteto-software-dados`

1. Configuração inicial do projeto

- ☐ Criar o projeto Electron com TypeScript (electron-vite, Electron Forge ou similar)
- ☐ Configurar tsconfig com strict: true e garantir que não exista .js de lógica
- ☐ Configurar ESLint/Prettier, scripts npm (dev, build, test) e .gitignore (.env, node_modules, banco)
- ☐ Criar a estrutura de pastas: main/, preload/, renderer/, shared/
- ☐ Escrever o README com instruções de instalação e execução

2. Modelagem do banco de dados

- ☐ Escolher o SGBD (SQLite com better-sqlite3 é o mais simples para desktop) e justificar
- ☐ Desenhar o diagrama ER (Mermaid ou draw.io) e salvar em docs/
- ☐ Tabelas: usuarios, clientes, categorias, fornecedores, equipamentos, reservas, itens_reserva, manutencoes, pagamentos, multas
- ☐ Definir chaves estrangeiras, ON DELETE/ON UPDATE e índices
- ☐ Criar scripts de migration (criação das tabelas) e seed (dados de exemplo)

3. Camada de acesso a dados (Repositórios)

- ☐ Classe de conexão com o banco (Database) com suporte a transações
- ☐ Classe base genérica de repositório (criar, buscarPorId, listar, atualizar, excluir)
- ☐ Um repositório por tabela, sempre com queries parametrizadas

4. Comunicação IPC (Main com Renderer)

- ☐ Arquivo shared/ipc-channels.ts com todos os canais no padrão dominio:acao
- ☐ DTOs tipados de entrada e saída em shared/dto/
- ☐ preload.ts expondo uma API tipada via contextBridge (window.api)
- ☐ Registro central dos handlers ipcMain.handle por domínio

5. Documentação técnica

- ☐ Documento de arquitetura em camadas (diagrama + explicação de cada camada)
- ☐ Dicionário de dados (tabela, campos, tipos e relacionamentos)

Pessoa 2 — Regras de Negócio e CRUD (Domínio POO)

Responsável: _____

Foco: Implementa o coração do sistema de aluguel em classes TypeScript: entidades, services, CRUD de todas as entidades e as regras da especificação.

Requisitos cobertos: Req. 2 (CRUD de todas as entidades), Req. 5 (lógica em classes)

Agente de IA de apoio: `regras-negocio-dominio`

1. Entidades de domínio (classes)

- ☐ Classes: Usuario, Cliente, Categoria, Fornecedor, Equipamento, Reserva, ItemReserva, Manutencao, Pagamento, Multa
- ☐ Enums: Modalidade (VENDA/LOCACAO), EstadoConservacao (NOVO/MUITO_BOM/MEDIO/REGULAR), StatusEquipamento, StatusReserva, TipoFornecedor
- ☐ Classes de erro de domínio (RegraNegocioError, EquipamentoIndisponivelError)

2. CRUD completo (services + handlers IPC)

- ☐ Clientes, Categorias, Fornecedores, Equipamentos
- ☐ Reservas e Itens de Reserva
- ☐ Manutenções, Pagamentos e Multas
- ☐ Usuários (junto com a Pessoa 3, por causa do hash de senha)

3. Inventário e disponibilidade

- ☐ Somente itens de LOCACAO podem ser reservados
- ☐ Verificação de disponibilidade por período (sem sobreposição de reservas e sem manutenção aberta)
- ☐ Atualização automática do status do equipamento (Disponível, Reservado, Em Manutenção, Indisponível)

4. Reservas, devoluções e multas

- ☐ Criar reserva com vários itens dentro de uma transação
- ☐ Ciclo de status: PENDENTE » ATIVA » DEVOLVIDA / CANCELADA (bloquear transições inválidas)
- ☐ Registrar devolução com o novo estado de conservação
- ☐ Cálculo automático de multa por atraso (diária fixa ou percentual — padrão Strategy)
- ☐ Listagem de devoluções pendentes e atrasadas

5. Financeiro e histórico

- ☐ Registro de pagamento do aluguel, caução (retida/devolvida) e quitação de multas
- ☐ Histórico do cliente: reservas ativas, pendências, pagamentos e multas

6. Fornecedores e manutenção

- ☐ Diferenciar fornecedor de COMPRA e prestador de MANUTENCAO
- ☐ Enviar equipamento para manutenção e registrar retorno (data, custo, novo estado)
- ☐ Validar vínculos: equipamento com fornecedor de compra; manutenção com prestador

7. Testes

- ☐ Testes unitários (Vitest/Jest) para disponibilidade, multas e transições de status

Pessoa 3 — APIs, Integrações e Segurança

Responsável: _____

Foco: Implementa as 4 integrações de API e toda a camada de segurança, incluindo login e controle de acesso.

Requisitos cobertos: Req. 3 (4 situações de API), Req. 4 (Segurança)

Agente de IA de apoio: `especialista-apis-integracoes + seguranca-dados-sensiveis`

1. Integração 1 — CEP (ViaCEP)

- ☐ Classe ViaCepClient no Main Process + canal IPC integracoes:buscarCep
- ☐ Preenchimento automático de rua, bairro, cidade e UF no cadastro de cliente
- ☐ Tratar CEP inválido, CEP inexistente e falta de internet (permitir digitação manual)

2. Integração 2 — Previsão do Tempo

- ☐ Classe ClimaClient (OpenWeatherMap ou Open-Meteo)
- ☐ Consultar o clima da data de retirada na confirmação da reserva
- ☐ Gerar alerta para categorias sensíveis ao clima (caiaque, bicicleta, prancha)

3. Integração 3 — Rotas / Mapas

- ☐ Classe RotasClient (OpenRouteService ou OSRM + Nominatim)
- ☐ Calcular distância e tempo até o cliente com devolução atrasada (busca presencial)

4. Integração 4 — Notificações / API própria

- ☐ API própria do grupo (Express em TypeScript ou mock) para lembretes de devolução
- ☐ Envio de lembrete/aviso de multa por e-mail (Mailtrap/Resend) ou WhatsApp simulado

5. Boas práticas das integrações

- ☐ Timeout com AbortController, cache em memória e mensagens de erro amigáveis
- ☐ Chaves de API em .env (fora do Git) + .env.example versionado
- ☐ Testes com respostas mockadas

6. Autenticação e controle de acesso

- ☐ Tela/fluxo de login; senhas com bcrypt (nunca em texto puro)
- ☐ Perfis ADMIN e FUNCIONARIO com permissões verificadas no Main Process
- ☐ Limite de tentativas de login e mensagem de erro genérica

7. Validação e endurecimento do Electron

- ☐ Validação de todos os dados recebidos via IPC (zod): CPF, CEP, e-mail, datas, valores
- ☐ contextIsolation, sandbox, nodeIntegration desligado e Content-Security-Policy
- ☐ Bloquear navegação externa e validar o remetente dos eventos IPC
- ☐ Proteção de dados sensíveis (LGPD): CPF mascarado e logs sem dados pessoais

8. Documento de segurança

- ☐ docs/SEGURANCA.md justificando cada decisão (exigido pelo professor)

Pessoa 4 — Interface (UI/UX) e Integração com o Electron

Responsável: _____

Foco: Constrói todas as telas da aplicação desktop e as conecta ao Main Process pela API exposta no preload (window.api).

Requisitos cobertos: Req. 2 (telas de CRUD), Req. 6 (Renderer em TypeScript)

Agente de IA de apoio: `interface-ui-electron`

1. Base visual

- [ ] Escolher a abordagem (TypeScript puro com classes ou React/Vue com Vite + TS)
- [ ] Tema em CSS (variáveis de cores e espaçamento), layout com menu lateral
- [ ] Roteador simples entre telas e componentes reutilizáveis (tabela, formulário, modal, toast)
- [ ] Estados de carregando, vazio e erro em todas as telas

2. Telas de acesso e painel

- [ ] Login
- [ ] Dashboard: reservas ativas, devoluções atrasadas, equipamentos em manutenção e multas em aberto

3. Telas de cadastro (CRUD)

- [ ] Inventário de equipamentos (filtros por categoria, modalidade, status e conservação)
- [ ] Categorias
- [ ] Clientes (CEP com preenchimento automático) e histórico do cliente
- [ ] Fornecedores (abas Compra / Manutenção)
- [ ] Usuários (somente ADMIN)

4. Fluxo de reservas e devoluções

- [ ] Assistente de reserva: cliente » período » equipamentos disponíveis » confirmação
- [ ] Card de previsão do tempo e alerta de clima na confirmação
- [ ] Painel de devoluções (ativas, atrasadas, devolvidas), com multa calculada e botão de rota
- [ ] Telas de pagamentos/caução e de manutenções (envio e retorno)

5. Qualidade da interface

- [ ] Confirmação antes de excluir e feedback após salvar
- [ ] Validação visual dos formulários
- [ ] Acessibilidade: labels, foco visível, contraste e navegação por teclado
- [ ] Respeitar a CSP (sem scripts inline e sem innerHTML com dados do usuário)

6. Empacotamento

- [ ] Gerar o instalador/executável (electron-builder ou Electron Forge)
- [ ] Capturas de tela para o relatório e a apresentação